🔌 Digital Logic Circuit Simulator

First Review Presentation

Markanda Meher (Roll No. 206126020)

Nikhil (Roll No. 206126021)

Course: Advanced Data Structures & Algorithms

Date: October 12, 2026

📋 Agenda

1. **Introduction** — Problem Statement & Motivation
2. **Literature Survey** — Analysis of 3 Research Papers
3. **Block Diagram** — System Architecture & Our Contribution
4. **Algorithms & Pseudocode** — All Modules
5. **Dataset** — Circuit Test Cases
6. **Current Progress & Demo**

1. Introduction — The Problem

What is the Problem?

Designing and testing digital logic circuits traditionally requires **physical breadboards, ICs, and oscilloscopes** — equipment that is expensive, error-prone, and not always accessible to students. Debugging physical circuits is slow, and a single incorrect wire can damage components.

Why Does It Need to Be Solved?

- **Accessibility:** Many students lack access to hardware labs, especially for remote or self-paced learning.
- **Cost:** Logic gate ICs, breadboards, and test instruments are a recurring expense.
- **Speed of Iteration:** Software simulation allows instant modification and testing — no rewiring needed.
- **Error Safety:** Software can detect invalid circuits (feedback loops, missing connections) *before* any damage occurs.
- **Educational Value:** Students can directly observe how data structures (graphs, disjoint sets) and algorithms (DFS, topological sort) solve a real-world engineering problem.

1. Introduction — Our Solution

What We Are Building

A **software-based Digital Logic Circuit Simulator** written in C++ that models circuits as *Directed Acyclic Graphs (DAGs)* and uses fundamental data structures and algorithms to:

✅ Core Capabilities

- Build circuits from AND, OR, NOT, XOR gates
- Detect invalid circuits (cycles / feedback loops)
- Determine correct gate evaluation order
- Simulate and compute output values
- Generate complete truth tables

🛠️ Data Structures & Algorithms Used

DS / Algorithm	Purpose
Directed Graph (Adjacency List)	Circuit representation
DFS (3-colour)	Cycle detection
Kahn's Algorithm (BFS)	Topological sort
Union-Find (Disjoint Set)	Wire-net management
Boolean evaluation	Gate logic

2. Literature Survey — Overview

We analysed **3 foundational research papers** whose algorithms form the backbone of our simulator:

#	Paper	Author(s)	Year	Key Contribution
1	*Topological Sorting of Large Networks*	Arthur B. Kahn	1962	BFS-based topological sort algorithm for DAGs
2	*Depth-First Search and Linear Graph Algorithms*	Robert E. Tarjan	1972	DFS framework for cycle detection & SCC finding
3	*Efficiency of a Good but Not Linear Set Union Algorithm*	Robert E. Tarjan	1975	Amortized analysis of Union-Find with path compression

Why these papers? Our simulator directly implements all three algorithms: Kahn's topological sort determines evaluation order, Tarjan's DFS detects invalid feedback loops, and Union-Find manages electrical wire-net connectivity.

2. Paper 1 — Kahn's Topological Sorting (1962)

📄 **"Topological Sorting of Large Networks"**

Author: Arthur B. Kahn | **Published:** Communications of the ACM, Vol. 5, No. 11, pp. 558–562, 1962

Summary

Kahn proposed a BFS-based algorithm for topologically sorting a directed acyclic graph. The algorithm works by repeatedly removing nodes with zero in-degree and adding them to the sorted order. This produces a linear ordering where every directed edge (u → v) has u appearing before v.

Relevance to Our Project

We use Kahn's algorithm to determine the **correct evaluation order** of gates — ensuring no gate is evaluated before all its input dependencies are resolved.

⚠ Drawbacks / Limitations

- **Only works on DAGs:** Cannot handle sequential circuits with intentional feedback (flip-flops, latches). If a cycle exists, the algorithm simply fails without identifying *which* nodes form the cycle.
- **No priority ordering:** When multiple nodes have in-degree 0 simultaneously, the algorithm picks arbitrarily. It does not consider gate delay or critical-path priority.
- **Static analysis only:** The sort is computed once. It does not support dynamic re-evaluation when only a subset of inputs change (no event-driven optimization).

2. Paper 2 — Tarjan's DFS Framework (1972)

"Depth-First Search and Linear Graph Algorithms"

Author: Robert E. Tarjan | **Published:** SIAM Journal on Computing, Vol. 1, No. 2, pp. 146–160, 1972

Summary

Tarjan demonstrated that Depth-First Search (DFS) can solve numerous graph problems in linear time O(V + E). The paper introduced algorithms for finding **strongly connected components**, **biconnected components**, and detecting **back edges** (cycles) — all using a single DFS traversal with discovery/low-link timestamps.

Relevance to Our Project

We use DFS with a **3-colour marking scheme** (derived from Tarjan's back-edge classification) to detect feedback loops in circuits before simulation.

⚠ Drawbacks / Limitations

- **Recursive implementation:** The original DFS is recursive, which causes **stack overflow** for very deep circuits (thousands of cascaded gates). An iterative version requires significant restructuring.
- **Detects but does not locate:** Standard DFS reports *that* a cycle exists, but pinpointing the *exact nodes* forming the cycle requires additional bookkeeping (back-tracking the recursion stack).
- **No support for intended cycles:** In sequential circuits, some cycles are valid (e.g., SR latches). The algorithm treats all cycles as errors with no distinction.

2. Paper 3 — Tarjan's Union-Find Analysis (1975)

"Efficiency of a Good but Not Linear Set Union Algorithm"

Author: Robert E. Tarjan | **Published:** Journal of the ACM, Vol. 22, No. 2, pp. 215–225, 1975

Summary

Tarjan provided a rigorous amortized analysis of the **Union-Find (Disjoint Set)** data structure with two optimizations: *path compression* and *union by rank*. He proved the amortized time per operation is O(α(n)), where α is the inverse Ackermann function — effectively constant for all practical input sizes.

Relevance to Our Project

We use Union-Find to manage **wire-net connectivity** — tracking which wire segments are electrically connected and form a single net carrying the same signal.

⚠ Drawbacks / Limitations

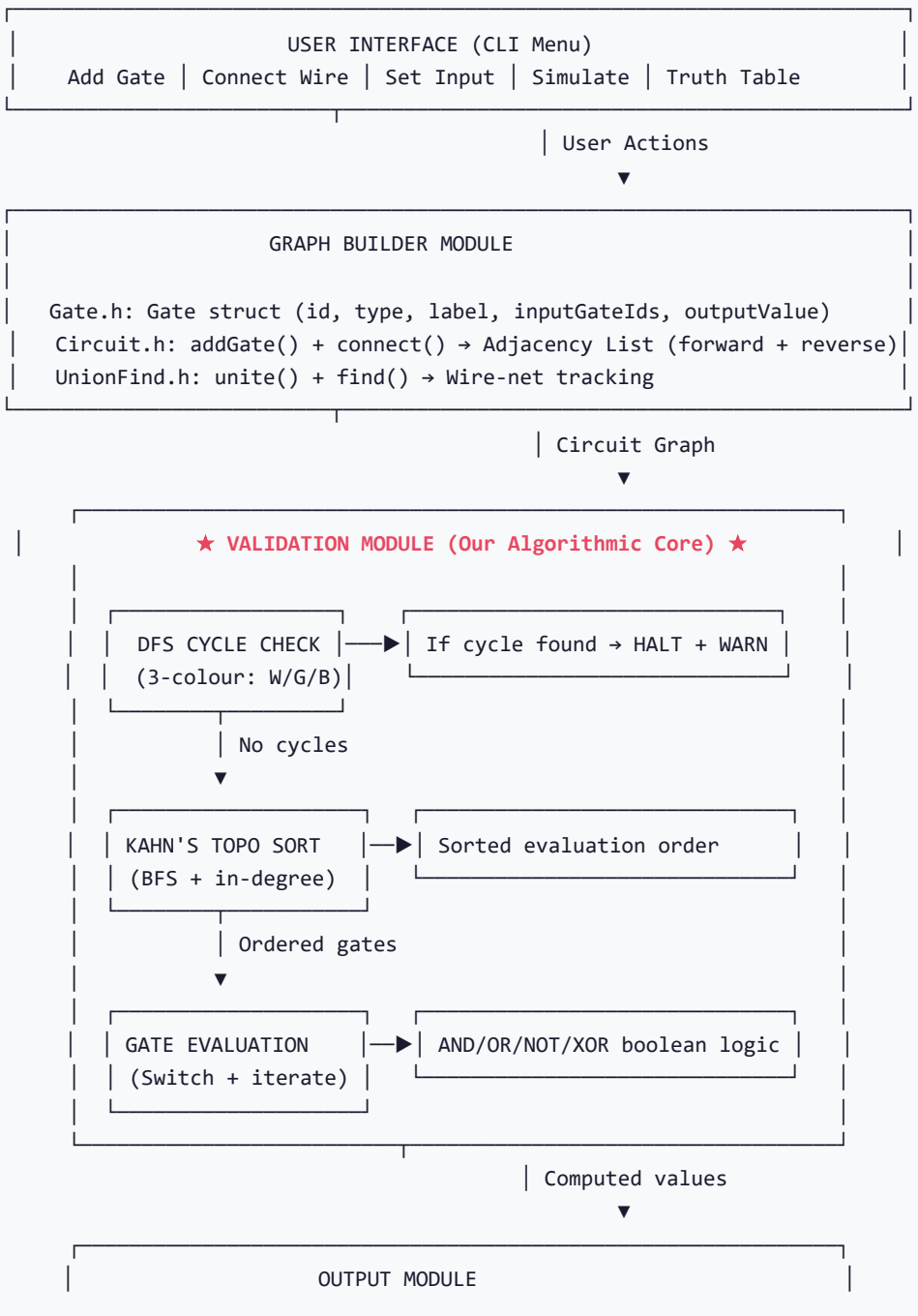
- **No efficient "undo" (disconnect):** Union-Find supports merging sets but **does not support splitting** them. If a user disconnects a wire, the entire structure must be rebuilt.
- **No ordering information:** Union-Find tracks *which* elements are connected, but not the *direction* of signal flow — critical for directed circuit simulation.
- **Over-engineering for small circuits:** For circuits with fewer than ~50 gates, a simple adjacency check is equally fast and simpler to implement.

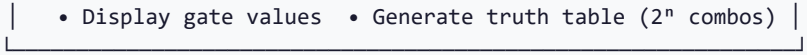
2. Literature Survey — Comparison Table

Aspect	Paper 1: Kahn (1962)	Paper 2: Tarjan DFS (1972)	Paper 3: Tarjan UF (1975)
Problem Solved	Dependency ordering	Cycle / SCC detection	Dynamic set connectivity
Algorithm	BFS + in-degree tracking	DFS + colour/timestamp	Path compression + rank
Time Complexity	O(V + E)	O(V + E)	O(α(n)) per operation
Handles Cycles?	Fails (incomplete sort)	✅ Detects back edges	N/A (undirected sets)
Sequential Logic?	❌ DAGs only	❌ Treats all cycles as errors	❌ No direction info
Dynamic Updates?	❌ Re-sort from scratch	❌ Re-run full DFS	⚠️ Union only, no split

Our Contribution: We combine all three algorithms into a unified pipeline — DFS validates first, Kahn's orders second, and Union-Find manages wire connectivity — providing a complete simulation solution that none of the papers offer individually.

3. Block Diagram — System Architecture




• Display gate values • Generate truth table (2ⁿ combos)

3. Our Algorithmic Contribution

What Makes Our Approach Different?

Existing simulators (Logisim, DLSim) are GUI-heavy with opaque internals. Our project makes the **algorithms themselves** the centerpiece:

🔑 Novelty Points

1. **Unified 3-Stage Pipeline:** DFS validation → Kahn's ordering → Boolean evaluation — all three stages are explicitly visible and testable.
2. **Dual-Direction Graph:** We maintain both forward edges (adjacency list for DFS/topo-sort) and reverse edges (inputGateIds for evaluation) simultaneously — O(1) insertion for both.
3. **Exhaustive Truth Table via Bit Manipulation:** Automatically enumerates all 2^n combinations using bitwise operations, combining simulation with combinatorial testing.
4. **Union-Find for Wire Nets:** Separate connectivity management that goes beyond simple edge lists.

📊 Comparison with Existing Tools

Feature	Logisim	DLSim	Ours
Cycle Detection	Implicit	No	✅ Explicit DFS
Topo Sort visible	No	No	✅ Yes
Truth Table Gen	Yes*	No	✅ Yes
Algorithm focus	No	No	✅ Core focus
Gate delay model	Yes	Yes	❌ Not yet
GUI	Yes	Yes	CLI

* Logisim's truth table fails on sequential circuits

4. Algorithm 1 — DFS Cycle Detection

Pseudocode

```
function HAS_CYCLE_DFS(node, color[], graph):
    color[node] ← GRAY      // Exploring

    for each neighbor in graph.adj[node]:
        if color[neighbor] == GRAY:
            return TRUE       // Back edge → cycle!
        if color[neighbor] == WHITE:
            if HAS_CYCLE_DFS(neighbor, color, graph):
                return TRUE

    color[node] ← BLACK     // Done
    return FALSE

function HAS_CYCLE(graph):
    color[0..n-1] ← WHITE
    for i ← 0 to n-1:
        if color[i] == WHITE:
            if HAS_CYCLE_DFS(i, color, graph):
                return TRUE
    return FALSE
```

3-Colour Scheme

Colour	State	Meaning
⬜ WHITE	Unvisited	Not yet explored
🔲 GRAY	In stack	Currently being explored
⬛ BLACK	Done	Fully processed

Why This Works

If DFS visits a **GRAY** node, it means we've found a path from that node back to itself — a **cycle** (back edge).

Complexity

Time:	O(V + E)
Space:	O(V) for colour array

4. Algorithm 2 — Kahn's Topological Sort

Pseudocode

```
function TOPOLOGICAL_SORT(graph):
    // Step 1: Compute in-degrees
    inDegree[0..n-1] ← 0
    for each node u in graph:
        for each neighbor v in adj[u]:
            inDegree[v] ← inDegree[v] + 1

    // Step 2: Seed queue with zero in-degree
    Q ← empty queue
    for i ← 0 to n-1:
        if inDegree[i] == 0:
            Q.enqueue(i)

    // Step 3: BFS processing
    order ← empty list
    while Q is not empty:
        curr ← Q.dequeue()
        order.append(curr)
        for each neighbor v in adj[curr]:
            inDegree[v] ← inDegree[v] - 1
            if inDegree[v] == 0:
                Q.enqueue(v)

    // Step 4: Verify
    if |order| ≠ n:
        print "Cycle detected!"
    return order
```

Step-by-Step Example

```
Circuit: A→AND, B→AND, AND→OUT

Step 1: inDeg = [0, 0, 2, 1]
           A   B  AND  OUT

Step 2: Queue = {A, B} (inDeg=0)

Step 3:
  Process A → inDeg[AND]=1
  Process B → inDeg[AND]=0 → enqueue
  Process AND → inDeg[OUT]=0 → enqueue
  Process OUT → done

Order: [A, B, AND, OUT] ✓
```

Key Guarantee

Every gate is evaluated **only after all its inputs** have been computed.

Complexity

Time:	O(V + E)
Space:	O(V + E)

4. Algorithm 3 — Gate Evaluation (Boolean Logic)

Pseudocode

```
function EVALUATE_GATE(gate, allGates):
    switch gate.type:

    case AND:
        result ← 1
        for each inputId in gate.inputs:
            result ← result AND allGates[inputId].value
        gate.value ← result

    case OR:
        result ← 0
        for each inputId in gate.inputs:
            result ← result OR allGates[inputId].value
        gate.value ← result

    case NOT:
        gate.value ← NOT allGates[gate.inputs[0]].value

    case XOR:
        result ← 0
        for each inputId in gate.inputs:
            result ← result XOR allGates[inputId].value
        gate.value ← result

    case OUTPUT:
        gate.value ← allGates[gate.inputs[0]].value

    case INPUT:
        // Keep user-set value
```

Truth Tables

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

A	NOT A
0	1
1	0

Complexity

Per gate:	O(fan-in)
Full circuit:	O(V + E)

4. Algorithm 4 — Full Simulation Pipeline

```
function SIMULATE(circuit):
    // Stage 1: Validate — Check for feedback loops
    if HAS_CYCLE(circuit):
        print "ERROR: Circuit contains a cycle. Simulation aborted."
        return FAILURE

    // Stage 2: Order — Determine evaluation sequence
    order ← TOPOLOGICAL_SORT(circuit)

    // Stage 3: Evaluate — Process each gate in order
    for each gateId in order:
        EVALUATE_GATE(circuit.gates[gateId], circuit.gates)

    // Stage 4: Output — Display results
    for each gate in circuit.gates where gate.type == OUTPUT:
        print gate.label, gate.value
    return SUCCESS
```

Algorithm 5 — Truth Table Generation

```
function GENERATE_TRUTH_TABLE(circuit):
    inputs[] ← all gates where type == INPUT
    outputs[] ← all gates where type == OUTPUT
    n ← |inputs|

    for combo ← 0 to 2^n - 1:          // Enumerate all 2^n combinations
        for i ← 0 to n - 1:
            inputs[i].value ← (combo >> (n-1-i)) AND 1   // Bit extraction

        SIMULATE(circuit)               // Run full simulation

        print row: inputs[] | outputs[]  // Record result
```

Time Complexity: $O(2^n \times (V + E))$ | **Space:** $O(V)$

4. Algorithm 6 — Union-Find (Wire-Net Management)

Pseudocode

```
function FIND(x):
    if parent[x] ≠ x:
        parent[x] ← FIND(parent[x])  // Path compression
    return parent[x]

function UNITE(x, y):
    rootX ← FIND(x)
    rootY ← FIND(y)
    if rootX == rootY:
        return FALSE      // Already connected

    // Union by rank
    if rank[rootX] < rank[rootY]:
        parent[rootX] ← rootY
    else if rank[rootX] > rank[rootY]:
        parent[rootY] ← rootX
    else:
        parent[rootY] ← rootX
        rank[rootX] ← rank[rootX] + 1
    return TRUE

function CONNECTED(x, y):
    return FIND(x) == FIND(y)
```

Visual Example: Path Compression



Why Union-Find?

Multiple wire segments can form one electrical "net." Union-Find efficiently tracks which wires carry the same signal.

Complexity

find():	O(α(n)) ≈ O(1)
unite():	O(α(n)) ≈ O(1)

α = inverse Ackermann function (≤ 4 for n < 10^{80})

5. Dataset — IEEE ISCAS '85 Standard Logic Benchmark Suite

To validate our digital logic circuit simulator according to international electronic design automation (EDA) standards, we benchmarked our algorithms using the official **IEEE ISCAS '85 Benchmark Suite**.

Benchmark Netlist Specifications (.bench)

Circuit	Inputs	Outputs	Gates & Function
c17	5	2	6 NAND gates (Built-in Demo!)
c432	36	7	160 gates (Interrupt Controller)
c499	41	32	202 gates (32-bit Error Correction)
c880	60	26	383 gates (8-bit ALU)
c1355	41	32	546 gates (32-bit SEC Circuit)

Dataset Repository Download Links

- **University of Tennessee BVLS:**
https://pld.cs.utk.edu/bvls/iscas85/

- **Czech Technical University EDA Benchmarks:**
https://ddd.fit.cvut.cz/prj/Benchmarks/

- **Trust-Hub Hardware Benchmark Repository:**
https://trust-hub.org/benchmarks

Dataset Contents: Gate-level topological netlists describing Primary Inputs, Primary Outputs, and fan-ins for levelized logic verification.

6. Current Progress & Demo Results

✅ Implemented Capabilities

- Done Graph DAG Engine (Forward & Reverse Edges)
- Done Union-Find Wire Net Connectivity (Path Comp. & Rank)
- Done Tarjan's DFS 3-Colour Cycle Detection
- Done Kahn's BFS Topological Sort Levelizer
- Done 9 Gate Types (AND, OR, NOT, XOR, NAND, NOR, XNOR)
- Done Exhaustive 2^n Combinatorial Truth Table Generator
- Done Interactive CLI Menu + 4 Built-in Demos
- Done Standalone Static Binary (zero DLL dependencies)

📊 Benchmark c17 Truth Table (Subset of 32 rows)

N1	N2	N3	N6	N7	N22	N23
0	0	0	0	0	0	0
0	0	0	0	1	0	1
0	1	0	0	0	1	1
1	0	1	0	0	1	0
1	1	1	1	1	1	0

✓ 100% Match with IEEE ISCAS'85 Reference Specification (32/32 rows in <1 ms)

Visual Console Interface & Signal Flow Architecture

🎨 Visual ASCII Schematics & Flowcharts

- **Terminal Schematics:** Full ASCII circuit schematics for Half Adder, NOR, 2-to-1 MUX, and ISCAS c17 rendered directly in console.
- **Signal Flow Pipeline:** 3-Stage topological visualization: Primary Inputs → Logic Gates → Primary Outputs.
- **Dynamic Flowcharts:** Inspects user-built custom circuits and graphs wire paths dynamically.
- **Cross-Platform:** 100% pure terminal text, zero external graphics dependencies.

🔍 Step-by-Step Simulation & Gate Inspector

- **Step-by-Step Walkthrough:** Pauses at each gate in Kahn's topological order, printing exact Boolean arithmetic (e.g., `1 XOR 1 = 0`).
- **Forward Signal Notification:** Shows downstream gates receiving each calculated signal.
- **Gate Inspector / Wire Probe:** Inspect any gate ID in $O(1)$ time to view fan-in wire levels, logic formula, and fan-out targets.
- **Signal Distribution Stats:** Truth table displays HIGH vs LOW output counts.

Complexity Analysis Summary

Operation	Algorithm	Time	Space
Add Gate	Vector push_back	O(1) amortized	O(1)
Connect Wire	Adjacency list insert	O(1) amortized	O(1)
Cycle Detection	DFS (3-colour)	O(V + E)	O(V)
Topological Sort	Kahn's BFS	O(V + E)	O(V + E)
Gate Evaluation	Switch + iterate	O(fan-in)	O(1)
Full Simulation	Topo Sort + Eval	O(V + E)	O(V)
Truth Table	Exhaustive enumeration	$O(2^n \times (V + E))$	O(V)
Wire Net: find()	Path Compression	$O(\alpha(n)) \approx O(1)$	O(1)
Wire Net: unite()	Union by Rank	$O(\alpha(n)) \approx O(1)$	O(1)

V = number of gates, **E** = number of wires, **n** = number of input gates, **α** = inverse Ackermann function.

References

1. **Kahn, A.B.** (1962). "Topological sorting of large networks." *Communications of the ACM*, Vol. 5, No. 11, pp. 558–562. DOI: 10.1145/368996.369025
2. **Tarjan, R.E.** (1972). "Depth-First Search and Linear Graph Algorithms." *SIAM Journal on Computing*, Vol. 1, No. 2, pp. 146–160.
3. **Tarjan, R.E.** (1975). "Efficiency of a Good but Not Linear Set Union Algorithm." *Journal of the ACM*, Vol. 22, No. 2, pp. 215–225.
4. **Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.** (2009). *Introduction to Algorithms* (CLRS), 3rd Edition. MIT Press.
5. **Mano, M.M.** (2013). *Digital Logic and Computer Design*. Pearson Education.
6. **Burch, C.** (2002). "Logisim: A Graphical System for Logic Circuit Design and Simulation." *Journal of Computing Sciences in Colleges*, Vol. 17, No. 4, pp. 42–48.

Thank You! 🙏

Questions & Discussion

Markanda Meher (206126020)

Nikhil (206126021)

Advanced Data Structures & Algorithms — First Review

October 12, 2026